Midterm Project- R Code and Outputs

Load in neccessary packages

```
library(tidymodels)
library(caret)
library(corrplot)
library(glmnet)
library(kableExtra)
library(earth)
library(dplyr)
```

Part I

Using this dataset, please help Researcher A build a prediction model for PFS, aiming to understand how demographic, tumor, and treatment characteristics influence progression-free survival in breast cancer patients.

To answer this question, your report should at least include the following:

Model Training: Provide a detailed description of the model training procedure and how you obtained the model. Your description should include sufficient detail so that another statistician can reproduce the same model. Points will be deducted if the details provided are insufficient to reproduce your results.

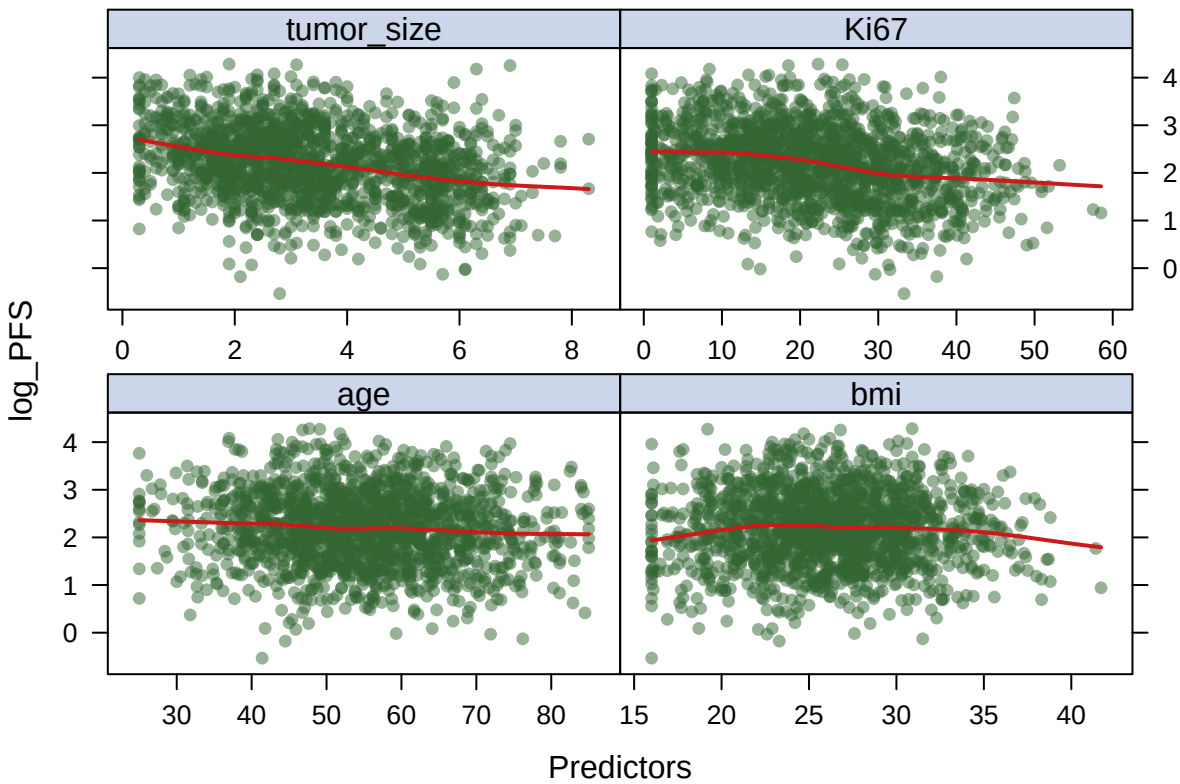
Import and partition data using 'tidymodels' package

```
load("data/datA.RData")
datA <- datA |>
  mutate(
    menopause = as.factor(menopause),
    race = as.factor(race),
    tumor_grade = as.factor(tumor_grade),
    ER = as.factor(ER),
    PR = as.factor(PR),
    HER2 = as.factor(HER2),
    stage = as.factor(stage),
    surgery = as.factor(surgery),
    chemo = as.factor(chemo),
    hormonal_therapy = as.factor(hormonal_therapy)
  )

set.seed(2)

datA_split = initial_split(datA, prop = 0.8)

# Extract the training and test data
training_datA = training(datA_split)
testing_datA = testing(datA_split)
```

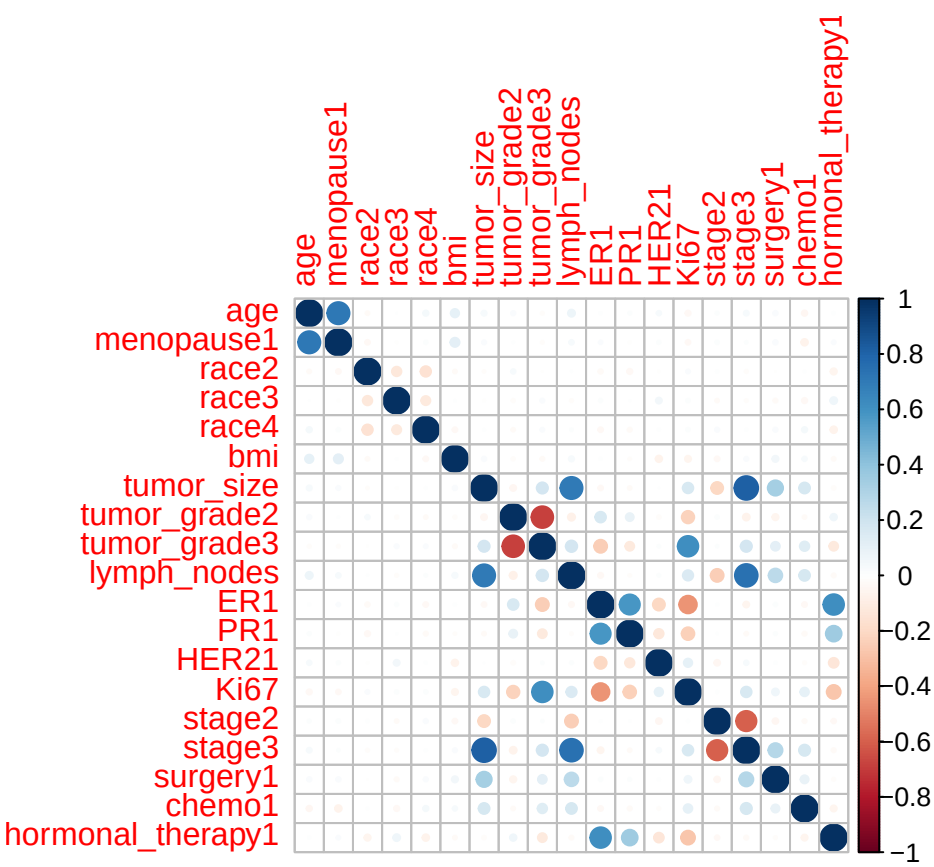
```r
# Training data
x_trainA = model.matrix(log_PFS ~ . - id, training_datA)[, -1]
y_trainA = training_datA$log_PFS

# Test data
x_testA = model.matrix(log_PFS ~ . - id, testing_datA)[, -1]
y_testA = testing_datA$log_PFS

# Remove ID
training_datA <- training_datA |>  select(-"id")
testing_datA  <- testing_datA  |> select(-"id")
```

Exploratory Data Analysis

```r
# Changing the plot display
theme1 <- trellis.par.get()
theme1$plot.symbol$col <- rgb(.2, .4, .2, .5)
theme1$plot.symbol$pch <- 16
theme1$plot.line$col <- rgb(.8, .1, .1, 1)
theme1$plot.line$lwd <- 2
theme1$strip.background$col <- rgb(.0, .2, .6, .2)
trellis.par.set(theme1)

# Feature plots to look at trends between predictors and the outcome (log_PFS)
featurePlot(x = training_datA[ ,c(1, 4, 5, 11)],
            y = training_datA[ ,16],
            plot = "scatter",
            span = .5,
            labels = c("Predictors","log_PFS"),
            type = c("p", "smooth"),
            layout = c(2, 2))
```




```r
# Calculates the correlation matrix for all the predictors
corrplot(cor(x_trainA), method = "circle", type = "full")
```



This data set includes 7 binary predictors, 3 categorical predictors, 5 continuous predictors, and 1 continuous outcome in log scale.

Chose elastic net because continuous variables formed a cloud, meaning we could assume linearity? homoscedasticity?

Correlation plot revealed that certain variables were highly correlated. Chose to address this with elastic net because we can address colinearity for [] and [] reasons

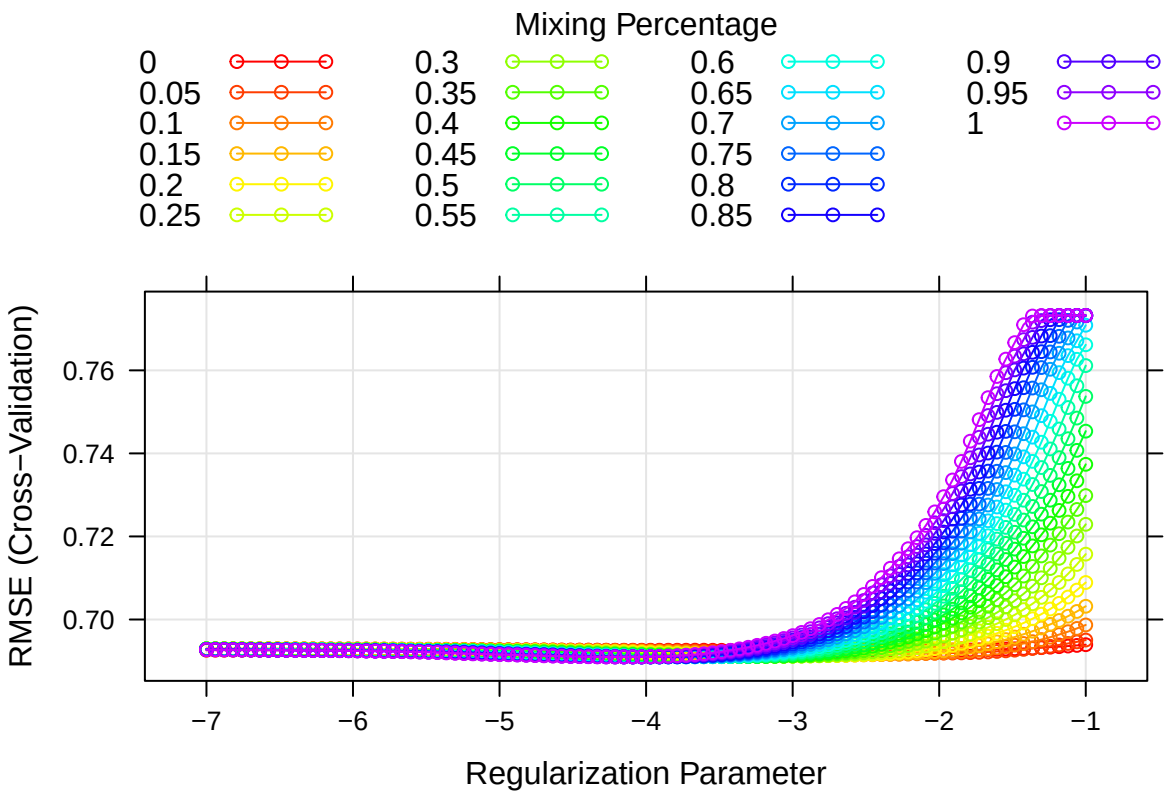
```
# Elastic Net

set.seed(2)
ctrl1 <- trainControl(method = "cv", number = 10) # set up cv
set.seed(2) # set seed for reproduction

enet.fit <- train(log_PFS ~ .,
                  data = training_datA,
                  method = "glmnet",
                  tuneGrid = expand.grid(alpha = seq(0, 1, length = 21),
                                         lambda = exp(seq(-1, -7, length = 100))),
                  trControl = ctrl1)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
enet.fit$bestTune
```

```
##      alpha     lambda
## 2047     1 0.01481491
```

```
myCol <- rainbow(25)
myPar <- list(superpose.symbol = list(col = myCol),
superpose.line = list(col = myCol))

plot(enet.fit, par.settings = myPar, xTrans = log)
```




```r
# Coefficients in the final model
coef(enet.fit$finalModel, enet.fit$bestTune$lambda)
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##                     s=0.01481491
## (Intercept)         2.6596549020
## age                -0.0025719864
## menopause1          .
## race2               .
## race3               0.0108623481
## race4              -0.0244504922
## bmi                 .
## tumor_size         -0.0545380491
## tumor_grade2        .
## tumor_grade3       -0.1057179919
## lymph_nodes        -0.0540172355
## ER1                 0.1709296973
## PR1                 .
## HER21              -0.1732033529
## Ki67               -0.0046400332
## stage2              .
## stage3              .
## surgery1           -0.0009660964
## chemo1              .
## hormonal_therapy1   0.1589522549
```

Results: Report the model. Interpret the model and assess its performance.

```r
# Predict on testing_datA
enet.pred <- predict(enet.fit, newdata = testing_datA)
```

```r
rmse_A <- RMSE(enet.pred, testing_datA$log_PFS)
r2_A  <- R2(enet.pred,   testing_datA$log_PFS)
```

```r
# Create table with the optimal tuning parameters for the chosen model
opt_para <- enet.fit$bestTune
rownames(opt_para) <- NULL
opt_para |>
  kable()
```

alpha	lambda
1	0.0148149

```r
# Create a table with the model coefficients
coefs1 <- as.data.frame(as.matrix(coef(enet.fit$finalModel, enet.fit$bestTune$lambda)))
coefs1$Predictors <- rownames(coefs1)
colnames(coefs1)[1] <- "Coefficients"
coefs1 <- coefs1[, c("Predictors", "Coefficients")]
rownames(coefs1) <- NULL
coefs1 |>
  kable()
```

Predictors	Coefficients
(Intercept)	2.6596549
age	-0.0025720
menopause1	0.0000000
race2	0.0000000
race3	0.0108623
race4	-0.0244505
bmi	0.0000000
tumor_size	-0.0545380
tumor_grade2	0.0000000
tumor_grade3	-0.1057180
lymph_nodes	-0.0540172
ER1	0.1709297
PR1	0.0000000
HER21	-0.1732034
Ki67	-0.0046400
stage2	0.0000000
stage3	0.0000000
surgery1	-0.0009661
chemo1	0.0000000
hormonal_therapy1	0.1589523

```r
# Create a table with the exponentiated coefficients
coefs2 <- as.data.frame(as.matrix(coef(enet.fit$finalModel, enet.fit$bestTune$lambda)))
coefs2$Predictors <- rownames(coefs2)
colnames(coefs2)[1] <- "Exponentiated Coefficients"
coefs2 <- coefs2[, c("Predictors", "Exponentiated Coefficients")]
coefs2$`Exponentiated Coefficients`<- exp(coefs2$`Exponentiated Coefficients`)
rownames(coefs2) <- NULL
```

```r
coefs2 |>
  kable()
```

Predictors	Exponentiated Coefficients
(Intercept)	14.2913563
age	0.9974313
menopause1	1.0000000
race2	1.0000000
race3	1.0109216
race4	0.9758460
bmi	1.0000000
tumor_size	0.9469225
tumor_grade2	1.0000000
tumor_grade3	0.8996783
lymph_nodes	0.9474158
ER1	1.1864073
PR1	1.0000000
HER21	0.8409666
Ki67	0.9953707
stage2	1.0000000
stage3	1.0000000
surgery1	0.9990344
chemo1	1.0000000
hormonal_therapy1	1.1722820

```r
# Create table with model performance results
tibble(
  "RMSE" = rmse_A,
  "R^2" = r2_A
) |>
  kable()
```

RMSE	R^2
0.7465867	0.1685384

Part II

About a decade later, Researcher B conducted a separate study enrolling 2000 breast cancer patients diagnosed between 2000 and 2005, using the same study design and variable definitions. Like Researcher A's study, all patients were followed until disease progression or death. Researcher B's data are stored in datB.RData. Researcher A would like to evaluate whether the prediction model built on the original data generalizes well to Researcher B's dataset. Does the model generalize well? If yes, provide justification. If not, build a new prediction model for the 2000–2005 cohort and discuss what has changed in prediction.

Import dataset and create matrix

```r
load("data/datB.RData")

datB <- datB |>
  mutate(
```

```r
    menopause = as.factor(menopause),
    race = as.factor(race),
    tumor_grade = as.factor(tumor_grade),
    ER = as.factor(ER),
    PR = as.factor(PR),
    HER2 = as.factor(HER2),
    stage = as.factor(stage),
    surgery = as.factor(surgery),
    chemo = as.factor(chemo),
    hormonal_therapy = as.factor(hormonal_therapy)
  )

# Remove id for matrix operations
all_datB <- datB |> select(-"id")
```

Evaluate the model's performance on researcher B's dataset

```r
# Use all of datB to test generalizability of elastic net model
enet_pred_B <- predict(enet.fit, newdata = all_datB)

# Compare performances
rmse_B <- RMSE(enet_pred_B, all_datB$log_PFS)
r2_B <- R2(enet_pred_B, all_datB$log_PFS)


# Report table comparing performance
performance_table <- data.frame(
  Dataset = c("datA (Internal Test Set)", "datB (External Validation)"),
  RMSE = c(round(rmse_A, 4), round(rmse_B, 4)),
  r2 = c(round(r2_A, 4), round(r2_B, 4))
)

colnames(performance_table) <- c("Dataset", "RMSE", "R^2")

performance_table |>
  kable(caption = "Elastic Net Model Performance: Internal vs External Validation") |>
  kable_styling(full_width = FALSE)
```

Table 5: Elastic Net Model Performance: Internal vs External Validation

Dataset	RMSE	R^2
datA (Internal Test Set)	0.7466	0.1685
datB (External Validation)	0.8363	0.1542

Creation of New Model

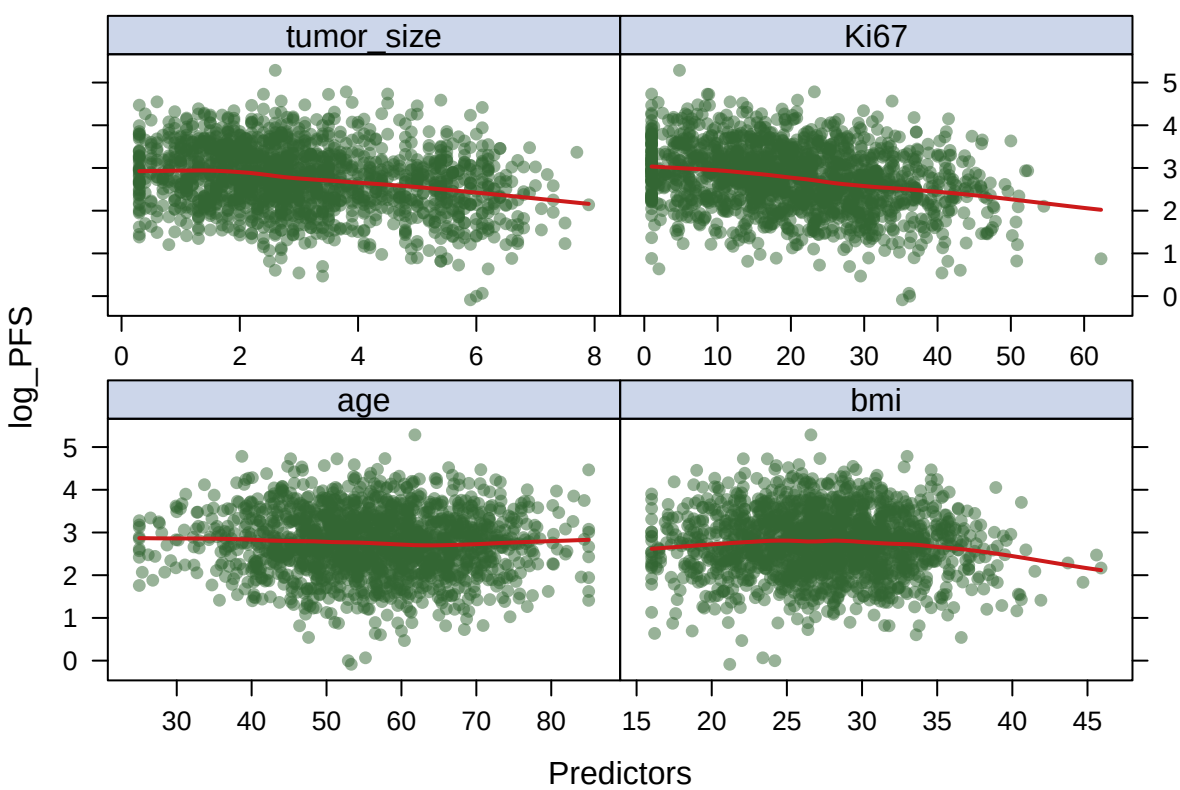
Import and partition data using 'tidymodels' package

```r
set.seed(2)

datB_split = initial_split(datB, prop = 0.8)
```

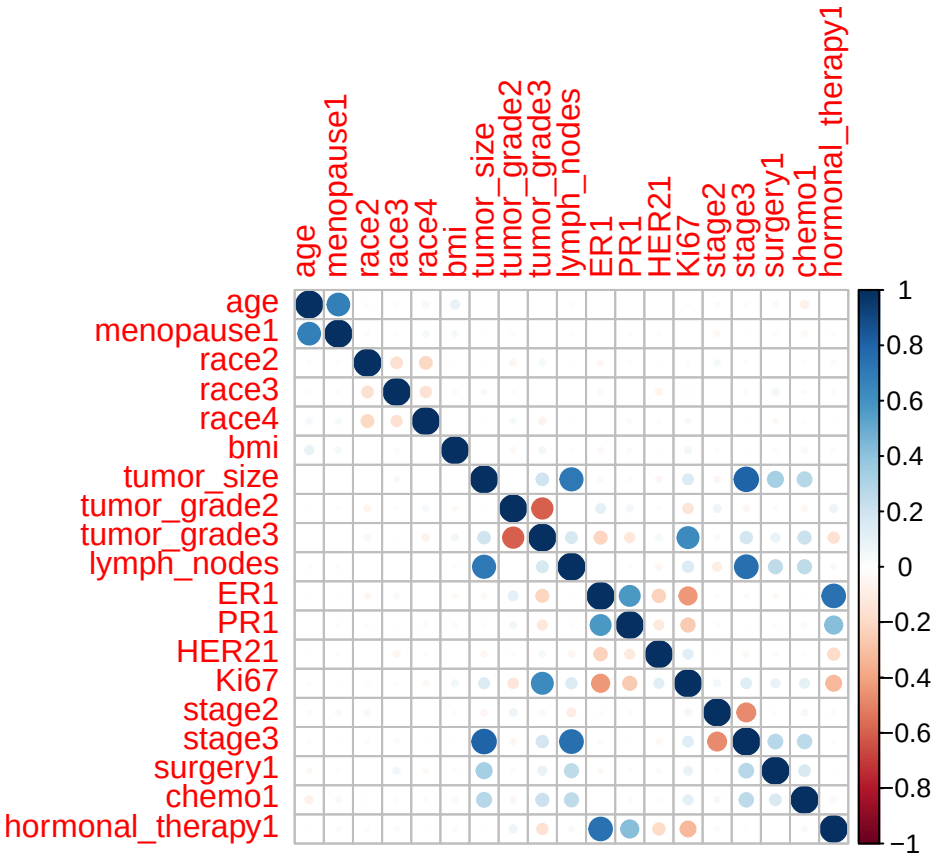
```r
# Extract the training and test data
training_datB = training(datB_split)
testing_datB = testing(datB_split)

# Training data
x_trainB = model.matrix(log_PFS ~ . - id, training_datB)[, -1]
y_trainB = training_datB$log_PFS

# Test data
x_testB  = model.matrix(log_PFS ~ . - id, testing_datB)[, -1]
y_testB = testing_datB$log_PFS

# Remove ID
training_datB <- training_datB |>  select(-"id")
testing_datB  <- testing_datB  |> select(-"id")
```

Exploratory Data Analysis

```r
# Feature plots to look at trends between predictors and the outcome (log_PFS)
featurePlot(x = training_datB[ ,c(1, 4, 5, 11)],
            y = training_datB[ ,16],
            plot = "scatter",
            span = .5,
            labels = c("Predictors","log_PFS"),
            type = c("p", "smooth"),
            layout = c(2, 2))
```



```r
# Calculates the correlation matrix for all the predictors
corrplot(cor(x_trainB), method = "circle", type = "full")
```



Elastic Net model

```r
# Elastic Net

set.seed(2)
ctrl1 <- trainControl(method = "cv", number = 10) # set up cv
set.seed(2) # set seed for reproduction

enet.test.b <- train(log_PFS ~ .,
                     data = training_datB,
                     method = "glmnet",
                     tuneGrid = expand.grid(alpha = seq(0, 1, length = 21),
                                            lambda = exp(seq(-1, -7, length = 100))),
                     trControl = ctrl1)
```

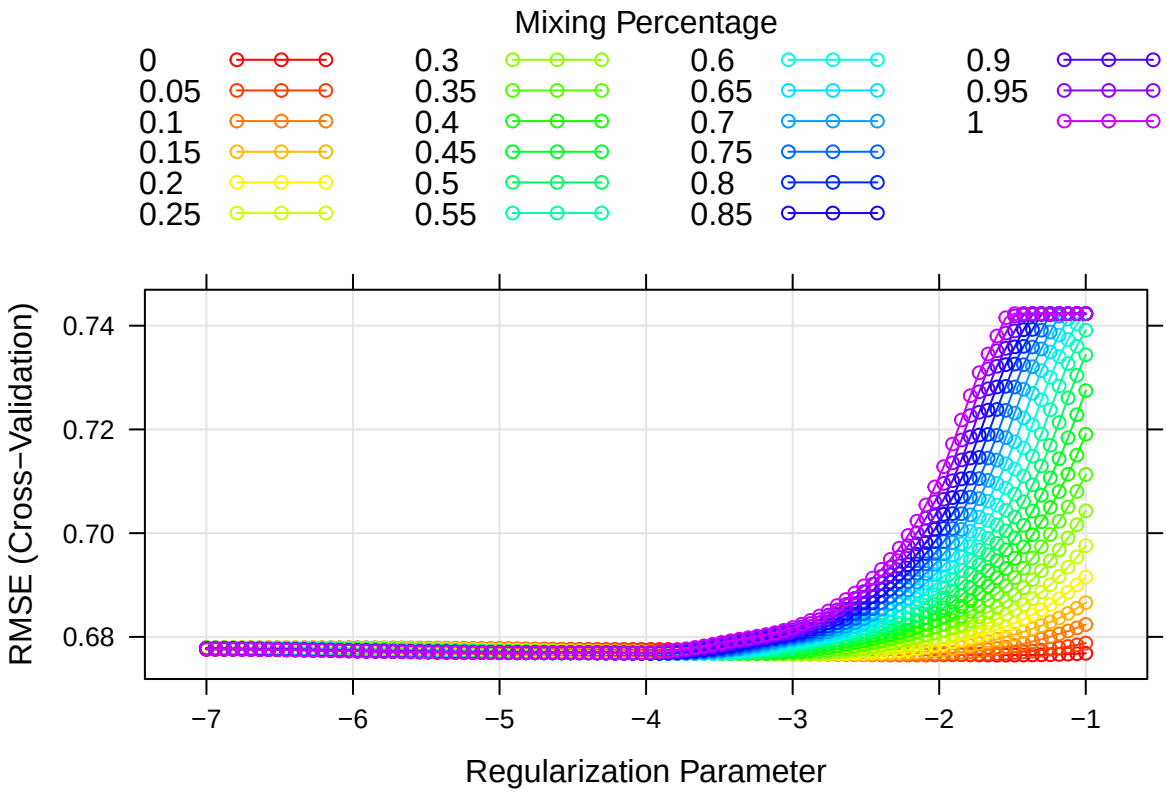
```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```r
enet.test.b$bestTune
```

```
##    alpha    lambda
## 91     0 0.2132149
```

```r
myCol <- rainbow(25)
```

```r
myPar <- list(superpose.symbol = list(col = myCol),
superpose.line = list(col = myCol))

plot(enet.test.b, par.settings = myPar, xTrans = log)
```




```r
# Coefficients in the final model
coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##                     s=0.2132149
## (Intercept)         2.998651952
## age                -0.002471616
## menopause1         -0.008133823
## race2              -0.040104815
## race3              -0.024429393
## race4               0.068951230
## bmi                -0.001545360
## tumor_size         -0.042193986
## tumor_grade2        0.074766605
## tumor_grade3       -0.106566952
## lymph_nodes        -0.016611495
## ER1                 0.144906908
## PR1                 0.096289175
## HER21               0.024744777
## Ki67               -0.004773993
## stage2              0.003120753
## stage3             -0.074148475
## surgery1           -0.025326354
## chemo1             -0.030902468
```

```
## hormonal_therapy1  0.165089624
```

```r
# Predict on testing_datB
enet.pred.b <- predict(enet.test.b, newdata = testing_datB)

rmse_testB <- RMSE(enet.pred.b, testing_datB$log_PFS)
r2_testB  <- R2(enet.pred.b, testing_datB$log_PFS)
```

```r
# Create table with the optimal tuning parameters for the chosen model
opt_para <- enet.test.b$bestTune
rownames(opt_para) <- NULL
opt_para |>
  kable()
```

alpha	lambda
0	0.2132149

```r
# Create a table with the model coefficients
coefs1 <- as.data.frame(as.matrix(coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)))
coefs1$Predictors <- rownames(coefs1)
colnames(coefs1)[1] <- "Coefficients"
coefs1 <- coefs1[, c("Predictors", "Coefficients")]
rownames(coefs1) <- NULL
coefs1 |>
  kable()
```

Predictors	Coefficients
(Intercept)	2.9986520
age	-0.0024716
menopause1	-0.0081338
race2	-0.0401048
race3	-0.0244294
race4	0.0689512
bmi	-0.0015454
tumor_size	-0.0421940
tumor_grade2	0.0747666
tumor_grade3	-0.1065670
lymph_nodes	-0.0166115
ER1	0.1449069
PR1	0.0962892
HER21	0.0247448
Ki67	-0.0047740
stage2	0.0031208
stage3	-0.0741485
surgery1	-0.0253264
chemo1	-0.0309025
hormonal_therapy1	0.1650896

```r
# Create a table with the exponentiated coefficients
coefs2 <- as.data.frame(as.matrix(coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)))
coefs2$Predictors <- rownames(coefs2)
colnames(coefs2)[1] <- "Exponentiated Coefficients"
```

```r
coefs2 <- coefs2[, c("Predictors", "Exponentiated Coefficients")]
coefs2$`Exponentiated Coefficients`<- exp(coefs2$`Exponentiated Coefficients`)
rownames(coefs2) <- NULL
coefs2 |>
  kable()
```

Predictors	Exponentiated Coefficients
(Intercept)	20.0584789
age	0.9975314
menopause1	0.9918992
race2	0.9606887
race3	0.9758666
race4	1.0713840
bmi	0.9984558
tumor_size	0.9586838
tumor_grade2	1.0776326
tumor_grade3	0.8989149
lymph_nodes	0.9835257
ER1	1.1559320
PR1	1.1010774
HER21	1.0250535
Ki67	0.9952374
stage2	1.0031256
stage3	0.9285338
surgery1	0.9749917
chemo1	0.9695701
hormonal_therapy1	1.1794988

```r
# Create table with model performance results
tibble(
  "RMSE" = rmse_testB,
  "R^2" = r2_testB
) |>
  kable()
```

RMSE	R^2
0.6555808	0.1645233

Compare coefficients from datA & datB model

```r
## Compare datA & datB model's exponentiated values

# extract coefficients from both models
coefs_A <- as.data.frame(as.matrix(coef(enet.fit$finalModel,
                                        enet.fit$bestTune$lambda)))
coefs_B <- as.data.frame(as.matrix(coef(enet.test.b$finalModel,
                                        enet.test.b$bestTune$lambda)))

# rename columns
colnames(coefs_A) <- "Coef_datA"
```

```r
colnames(coefs_B) <- "Coef_datB"

# add predictor names
coefs_A$Predictor <- rownames(coefs_A)
coefs_B$Predictor <- rownames(coefs_B)

# merge both into one table
coef_comparison <- merge(coefs_A, coefs_B, by = "Predictor", all = TRUE)

# exponentiate both and compute difference on exponentiated scale
coef_comparison$Exp_datA   <- round(exp(coef_comparison$Coef_datA), 4)
coef_comparison$Exp_datB   <- round(exp(coef_comparison$Coef_datB), 4)
coef_comparison$Difference <- round(coef_comparison$Exp_datB - coef_comparison$Exp_datA, 4)

# sort by absolute difference
coef_comparison <- coef_comparison[order(abs(coef_comparison$Difference),
                                         decreasing = TRUE), ]
rownames(coef_comparison) <- NULL

# keep only exponentiated columns
coef_comparison <- coef_comparison[, c("Predictor", "Exp_datA", "Exp_datB", "Difference")]

# display table
coef_comparison |>
  kable(caption = "Comparison of Elastic Net Coefficients: datA vs datB",
        col.names = c("Predictor",
                      "datA (Exp. Coefficient)",
                      "datB (Exp. Coefficient)",
                      "Difference"))
```

Table 10: Comparison of Elastic Net Coefficients: datA vs datB

Predictor	datA (Exp. Coefficient)	datB (Exp. Coefficient)	Difference
(Intercept)	14.2914	20.0585	5.7671
HER21	0.8410	1.0251	0.1841
PR1	1.0000	1.1011	0.1011
race4	0.9758	1.0714	0.0956
tumor_grade2	1.0000	1.0776	0.0776
stage3	1.0000	0.9285	-0.0715
race2	1.0000	0.9607	-0.0393
lymph_nodes	0.9474	0.9835	0.0361
race3	1.0109	0.9759	-0.0350
ER1	1.1864	1.1559	-0.0305
chemo1	1.0000	0.9696	-0.0304
surgery1	0.9990	0.9750	-0.0240
tumor_size	0.9469	0.9587	0.0118
menopause1	1.0000	0.9919	-0.0081
hormonal_therapy1	1.1723	1.1795	0.0072
stage2	1.0000	1.0031	0.0031
bmi	1.0000	0.9985	-0.0015
tumor_grade3	0.8997	0.8989	-0.0008
Ki67	0.9954	0.9952	-0.0002
age	0.9974	0.9975	0.0001